

CSCC24 – Principles of Programming Languages

Anya Tafliovich¹

¹with thanks to S.McIlraith, G.Penn, P.Ragde

specifying syntax informally

Example:

“Everything between `/*` and `*/` is a comment and should be ignored.”

```
/* Do such and such, watching out for problems.  
   Store the result in y. */  
x = 3;  */  
y = x * 17.2;
```

When syntax is defined informally, incompatible dialects of the language may evolve.

specifying PL syntax

- The state of the art is to define programming language syntax formally.
- There are a number of well-understood formalisms for doing so. We'll talk about this in some detail.
- Lexical rules specify the form of the building blocks of the language:
 - what's a token (keyword, identifier, literal, operator, punctuation, possibly white space)
 - how tokens are delimited
 - where white space can go
 - how comments are specified
- Syntax rules specify how to put the building blocks together.

grammars

Informal idea of grammar: a bunch of rules. For example:

- Don't end a sentence with a preposition.
- Subject and verb must agree in number.

A formal grammar is a somewhat different concept.

A language is a set of strings. A grammar generates a language — it specifies which strings are in the language.

There are many kinds of formal grammar.

Chomsky's Hierarchy

There are several categories of grammar, ordered by expressiveness (the last one is the most expressive):

- Regular Grammars
- Context-Free Grammars
- Context-Sensitive Grammars
- Phrase-Structure Grammars

This hierarchy (circa 1950) is named after linguist (and later political activist) Noam Chomsky, who researched grammars for natural languages.

Look him up!

Regular Expressions

Examples:

- $(0 + 1)^*$
- $1^*(: + ;)^*$
- $(a + b)^* aa(a + b)^*$

Notation:

- Kleene star: $*$ superscript denotes 0 or more repetitions.
- alternation: binary “+” denotes choice.
It is sometimes denoted by $|$, i.e., $(0|1)^*$.
- grouping: “(” and “)” are used for grouping
- empty string: ϵ (epsilon) denotes the empty or “null” string.
- empty language: $\emptyset$ denotes the language with *no* strings.

Regular Expressions

Formally, let Σ be a given finite alphabet. A string is a regular expression (RE) if and only if it can be derived from the primitive regular expressions:

- $\emptyset$
- ϵ
- any a , s.t. $a \in \Sigma$

by a finite number of applications of the following rules:

If r_1 and r_2 are REs, then so are:

- $r_1 + r_2$, or $r_1 | r_2$ (union, alternation)
- $r_1 r_2$, or $r_1 \cdot r_2$ (concatenation)
- r_1^* (Kleene closure)
- (r_1) (grouping)

Σ^* denotes the set of all finite strings over the alphabet Σ .

Regular Expressions

Given a RE r over an alphabet Σ , a language associated with r , denoted $L(r)$, is defined by:

- $L(\emptyset)$ is the empty language (language that contains no strings)
- $L(\epsilon)$ is the language $\{\epsilon\}$
- for $a \in \Sigma$, $L(a)$ is the language $\{a\}$

...

Regular Expressions

Given a RE r over an alphabet Σ , a language associated with r , denoted $L(r)$, is defined by:

...

If r_1 and r_2 are regular expressions, then

- $L(r_1 + r_2) = L(r_1) \cup L(r_2)$,
where $S \cup T = \{s \mid s \in S \text{ or } s \in T\}$
- $L(r_1 r_2) = L(r_1)L(r_2)$,
where $ST = \{st \mid s \in S, t \in T\}$
- $L((r_1)) = L(r_1)$
- $L(r_1^*) = (L(r_1))^*$,
where S^* denotes the smallest superset of S that contains ϵ and is closed under string concatenation

The precedence is (from highest to lowest) parentheses, Kleene star, concatenation, alternation.

Regular Expressions

A regular language is a language that can be expressed by a regular expression.

A regular language is a language that is accepted by a finite-state automaton.

Deterministic finite state automata (DFSA) and non-deterministic finite state automata (NFSA) accept the same class of languages, but DFSA can be exponentially larger.

We won't talk about automata in this course: you studied them in cscb36/... .

Regular Expressions — examples

Give regular expressions for these languages:

1. All alphanumeric strings beginning with an upper-case letter.

$$(A + B + \dots + Z)(A + B + \dots + Z + a + b + \dots + z + 0 + 1 + \dots + 9)^*$$

2. All strings of a's and b's in which the third-last character is b.

$$(a + b)^* b (a + b)(a + b)$$

3. All strings of 0's and 1's in which every pair of adjacent 0's appears before any pairs of adjacent 1's.

$$(1 + \epsilon)(0 + 01)^*(1 + 10)^*$$

4. All binary numbers with exactly three 1's.

$$0^* 1 0^* 1 0^* 1 0^*$$

limitations of Regular Expressions

Regular expressions are not powerful enough to describe some languages.

Examples:

- The language consisting of all strings of one or more a's followed by the same number of b's.
- The language consisting of strings containing a's, b's, left brackets, and right brackets, such that the brackets match.

Question: How can we be sure there is no regular expression for these languages?

Context-Free Grammars

CFGs are more powerful than regular expressions.

A CFG has four parts:

- A set of terminals:
the atomic symbols of the language.
- A set of non-terminals:
“variables” used in the grammar.
- A special non-terminal chosen as the starting non-terminal or start symbol:
represents the top-level construct of the language.
- A set of rules (or productions), each specifying one legal way that a non-terminal can be rewritten to a sequence of terminals and non-terminals.

Context-Free Grammars — example

A CFG for decimalss:

- Terminals: 0 1 2 3 4 5 6 7 8 9 .
- Non-terminals: decimal, part, digit.
- Productions:
 - A digit is any single terminal except ".".
 - A part is a digit.
 - A part is a digit followed by a part.
 - A decimal is a part, followed by ".", followed by a part.
- Start symbol: decimal.

Note that we use recursion to specify repeated occurrences.

We have “defined” this CFG using plain English. Not a very good idea.

Context-Free Grammars

A context-free grammar G is a tuple $\{\Sigma, V, S, P\}$, where

- Σ is the set of terminals
- V is the set of non-terminals
- $\Sigma \cap V = \emptyset$
- S is the start symbol, $S \in V$
- P is the set of production rules,
where each rule has the form
 $X \rightarrow w$ (sometimes written $X \Rightarrow w$ or $X ::= w$), where

$X \in V$	(X is a single non-terminal), and
$w \in (\Sigma \cup V)^*$	(w is a string of terminals and/or non-terminals)

Context-Free Grammars — example

Σ : {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, .}

V : {<decimal>, <part>, <digit>}, S : <decimal>

P :

```
<decimal> --> <part> . <part>
<part>      --> <digit>
<part>      --> <digit> <part>
<digit>     --> 0
<digit>     --> 1
<digit>     --> 2
<digit>     --> 3
...
<digit>     --> 9
```

Not very convenient. Shorthand: $X \rightarrow u|v|w$

```
<decimal> --> <part> . <part>
<part>    --> <digit> | <digit> <part>
<digit>   --> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```


Context-Free Grammars

A production rule $X \rightarrow w$ can be applied to a string of the form sXt , $s, t \in (V \cup \Sigma)^*$ to produce a string swt .

A grammar G generates a string s if there exists a (finite) sequence of applications of production rules, so that the first production rule in this sequence is applied to a start symbol, and the last production rule in the sequence produces s , $s \in \Sigma^*$. We write $G \Rightarrow^* s$.

The language generated by a grammar G is denoted $L(G)$, that is $L(G) = \{s \mid G \Rightarrow^* s\}$.

Convention: A start symbol is the first non-terminal listed.

Context-Free Grammars

CFGs are more “powerful” than REs. That is, there are languages that cannot be described with a RE but can be described with a CFG.

Example: The language consisting of strings with one or more a's followed by the same number of b's.

There is no regular expression for this language. (Why?)

CFG for the language:

$\langle AB \rangle \rightarrow ab \mid a \langle AB \rangle b$

Regular Grammars

- More restricted than CFGs.
- Limited to productions of the form:
 - Left-recursive grammar: at most one non-terminal and it appears as the left-most symbol in the string on the RHS of the production rule.
 $\langle S \rangle ::= \langle T \rangle a b$
 $\langle T \rangle ::= a \mid \langle T \rangle b$
 - Right-recursive grammar: at most one non-terminal and it appears as the right-most symbol in the string on the RHS of the production rule.
 $\langle S \rangle ::= a \langle T \rangle$
 $\langle T \rangle ::= b \langle T \rangle \mid a b$

Extended BNF

CFGs we've seen so far are in Backus-Naur Form (BNF).

There are extensions to BNF that make it more concise, but no more powerful (*i.e.*, there is no language that can be expressed with EBNF but not with BNF). Some examples:

- $\{ blah \}$ denotes zero or more repetitions of *blah*.
- $[blah]$ denotes that *blah* is optional.
- a^+ denotes one or more repetitions.
- a^n denotes a maximum number of repetitions.
- $()$ are used for grouping.

There is no one standard EBNF; it just refers to any extension of BNF.

Exercise: Show that the above operators do not add any expressive power.

derivations

A derivation of string s is the sequence of applications of production rules that generates s .

For example, the grammar

```
<decimal> --> <part> . <part>
<part>      --> <digit> | <digit> <part>
<digit>     --> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

generates the string 97.123.

A string is in the language generated by a grammar iff there is a derivation for it.

Derivation of 97.123:

```
<dec>
--> <p>.<p> --> <d><p>.<p> --> <d><d>.<p> --> <d><d>.<d><p>
--> <d><d>.<d><d><p> --> <d><d>.<d><d><d> --> 9<d>.<d><d><d>
--> 97.<d><d><d> --> 97.1<d><d> --> 97.12<d> --> 97.123
```

parse trees

A parse tree is a tree in which

- the root is the start symbol,
- every leaf is a terminal, and
- every internal node is a non-terminal, and its children correspond, in order, to the RHS of one of its productions in the grammar.

Parse trees show the structure within a string of the language.

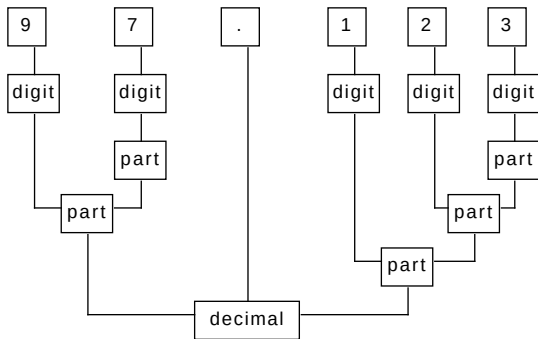
Parsing is the process of producing a parse tree.

A string is in the language generated by a grammar iff there is a parse tree for the string.

Parse tree for 97.123:

parse trees

Parse tree for 97.123:



syntactic ambiguity — in English

Syntactically ambiguous sentences of English:

- “I saw the dog with the binoculars.”
- “The friends you praise sometimes deserve it.”
- “They seemed nice to her.”

Aside: We can often “disambiguate” ambiguous sentences. How?

syntactic ambiguity — in a programming language

Example (grammar excerpt):

```
<stmt>      --> <assnt-stmt> | <loop-stmt> | <if-stmt>
<if-stmt> --> if <boolean-expr> then <stmt>
              | if <boolean-expr> then <stmt> else <stmt>
```

Example sentence:

```
if is_odd(x) then
if x == 1 then
print "foo";
else
print "bar";
```

Exercise: Draw the two parse trees.

syntactic ambiguity — definition

A grammar is ambiguous iff it generates a string for which there are two or more distinct parse trees.

A string is ambiguous with respect to a grammar iff that grammar generates two or more distinct parse trees for the string.

Note that having two distinct *derivations* does not make a string ambiguous.

A derivation corresponds to a traversal through a parse tree, and one can traverse a single tree in many orders.

syntactic ambiguity — example

Grammar: if statement two slides ago.

Sentence:

```
if is_odd(x) then  
  print "foo";
```

One parse tree, two derivations.

Exercise: provide the parse tree and two derivations.

syntactic ambiguity

When specifying a programming language, we want the grammar to be completely unambiguous.

Question: Is there a procedure one can follow to determine whether or not a given grammar is ambiguous?

We have two strategies for dealing with ambiguity:

1. Change the language to include delimiters.
2. Change the grammar to impose associativity and precedence.

dealing with ambiguity — delimiters

```
<stmt>    --> <assnt-stmt> | <loop-stmt> | <if-stmt>
<if-stmt> --> if <boolean-expr> then <stmt> fi
           |  if <boolean-expr> then <stmt>
                               else <stmt>
                               fi
```

Now the ambiguous program from the previous slide is no longer in the language, and the programmer is forced to disambiguate.

example: a CFG for arithmetic expressions

Grammar G1:

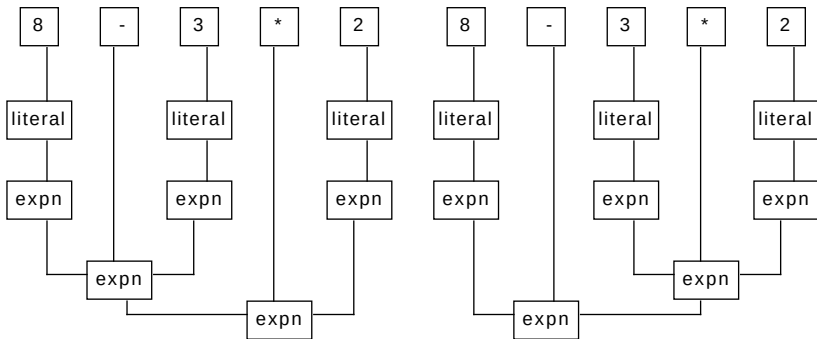
```
<expn> --> <expn> + <expn> |  
           <expn> - <expn> |  
           <expn> * <expn> |  
           <expn> / <expn> |  
           <expn> ^ <expn> |  
           (<expn>)      |  
           <identifier>  |  
           <literal>
```

The grammar is ambiguous.

Example: parse $8 - 3 * 2$.

example: a CFG for arithmetic expressions

Example: parse $8 - 3 * 2$



changing the language to include delimiters

Grammar G_2 :

```
<expn> --> (<expn> + (<expn> |  
            (<expn> - (<expn> |  
            (<expn> * (<expn> |  
            (<expn> / (<expn> |  
            (<expn> ^ (<expn> |  
            (<expn> |  
            <identifier> |  
            <literal>
```

With the new restrictions, we have:

$$(8) - ((3) * (2)) \in L(G_2)$$

$$((8) - (3)) * (2) \in L(G_2)$$

$$8 - 3 * 2 \notin L(G_2)$$

changing the grammar to impose precedence

- Low precedence: addition $+$ and subtraction $-$.
- Medium precedence: multiplication $*$ and division $/$.
- Higher precedence: exponentiation $^$.
- Highest precedence: parenthesized expressions ($\langle \text{expr} \rangle$).

Approach: introduce a non-terminal for every precedence level.

changing the grammar to impose precedence

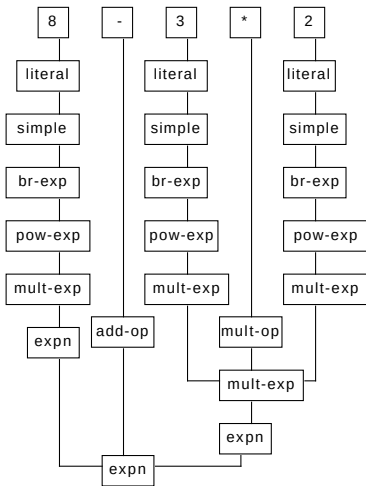
Grammar G3:

```
<expn>      --> <expn> <add-op> <expn> | <mult-exp>
<mult-exp>  --> <mult-exp> <mult-op> <mult-exp> | <pow-exp>
<pow-exp>   --> <pow-exp> ^ <pow-exp> | <br-exp>
<br-exp>    --> (<expn>) | <simple>
<simple>     --> <identifier> | <literal>
<add-op>    --> + | -
<mult-op>   --> * | /
```

Grouping in parse tree now reflects precedence.

changing the grammar to impose precedence

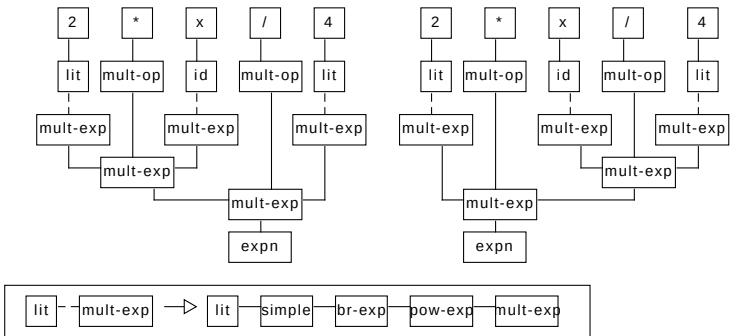
Grouping in parse tree now reflects precedence.



example: a CFG for arithmetic expressions

We still have ambiguity! Example: $2 * x / 4$ is still a problem.

- The grouping of operators of same precedence is not disambiguated.
- For non-commutative operators, only one parse tree is correct.
- Operators may be left or right associative.



associativity

- Deals with operators of same precedence.
- Implicit grouping or parenthesizing.
- Left associative: $*$, $/$, $+$, $-$.
- Right associative: $^$.

Approach: For left-associative operators, put the recursive term before the non-recursive term in a production rule. For right-associative operators, put it after.

changing the grammar to impose associativity

Grammar G4:

```
<expn>      --> <expn> <add-op> <mult-exp> | <mult-exp>
<mult-exp>  --> <mult-exp> <mult-op> <pow-exp> | <pow-exp>
<pow-exp>   --> <br-exp> ^ <pow-exp> | <br-exp>
<br-exp>    --> (<expn>) | <simple>
<simple>     --> <identifier> | <literal>
<add-op>    --> + | -
<mult-op>   --> * | /
```

